

DAA All Experiment Codes

Merge Sort

```
#include<stdio.h>

void merge(int A[], int low, int mid, int high){

    int l = mid - low + 1;
    int r = high - mid;

    int L[l], R[r];

    for(int i = 0; i < l; i++){
        L[i] = A[low + i];
    }

    for(int i = 0; i < r; i++){
        R[i] = A[mid + 1 + i];
    }

    int i = 0, j = 0, k = low;

    while(i < l && j < r){

        if(L[i] <= R[j]){
            A[k] = L[i];
            i++;
        }
        else{
            A[k] = R[j];
            j++;
        }

        k++;
    }

    while(i < l){
        A[k] = L[i];
        i++;
        k++;
    }

    while(j < r){
        A[k] = R[j];
        j++;
        k++;
    }
}

void mergeSort(int A[], int low, int high){

    if(low < high){

        int mid = (low + high)/2;

        mergeSort(A, low, mid);
        mergeSort(A, mid+1, high);

        merge(A, low, mid, high);
    }
}
```

Quick Sort

```
#include<stdio.h>

int partition(int A[], int low, int high){

    int pivot = A[low];
    int i = low;

    for(int j = low+1; j <= high; j++){
        if(A[j] < pivot){
```

```

        i++;

        int temp = A[j];
        A[j] = A[i];
        A[i] = temp;
    }

    int temp = A[i];
    A[i] = A[low];
    A[low] = temp;

    return i;
}

void quickSort(int A[], int low, int high){
    if(low < high){
        int p = partition(A, low, high);

        quickSort(A, low, p-1);
        quickSort(A, p+1, high);
    }
}

```

Binary Search

```

#include<stdio.h>

int binarySearch(int A[], int low, int high, int goal){
    if(low <= high){
        int mid = (low + high)/2;

        if(A[mid] == goal){
            return mid;
        }

        else if(goal < A[mid]){
            return binarySearch(A, low, mid-1, goal);
        }

        else{
            return binarySearch(A, mid+1, high, goal);
        }
    }

    return -1;
}

```

Fibonacci DP

```

#include<stdio.h>

int fib(int n, int dp[]){
    if(n <= 1){
        return n;
    }

    if(dp[n] != -1){
        return dp[n];
    }

    dp[n] = fib(n-1, dp) + fib(n-2, dp);

    return dp[n];
}

```

0/1 Knapsack

```

#include<stdio.h>

#define MAX 100

int max(int a, int b){
    if(a > b){
        return a;
    }

    return b;
}

int knapsack(int W, int n, int w[], int val[], int dp[][MAX]){
    if(W == 0 || n == 0){
        return 0;
    }

    if(dp[W][n] != -1){
        return dp[W][n];
    }

    if(w[n-1] > W){
        dp[W][n] = knapsack(W, n-1, w, val, dp);
        return dp[W][n];
    }

    dp[W][n] = max(
        val[n-1] + knapsack(W-w[n-1], n-1, w, val, dp),
        knapsack(W, n-1, w, val, dp)
    );

    return dp[W][n];
}

```

LCS

```

#include<stdio.h>

#define MAX 100

int max(int a, int b){
    if(a > b){
        return a;
    }

    return b;
}

int lcs(char X[], char Y[], int i, int j, int dp[][MAX]){
    if(i == 0 || j == 0){
        return 0;
    }

    if(dp[i][j] != -1){
        return dp[i][j];
    }

    if(X[i-1] == Y[j-1]){
        dp[i][j] = 1 + lcs(X, Y, i-1, j-1, dp);
        return dp[i][j];
    }

    dp[i][j] = max(
        lcs(X, Y, i-1, j, dp),
        lcs(X, Y, i, j-1, dp)
    );

    return dp[i][j];
}

```

Matrix Chain Multiplication

```
#include<stdio.h>

#define MAX 100
#define INF 999999

int min(int a, int b){
    if(a < b){
        return a;
    }
    return b;
}

int mcm(int p[], int i, int j, int dp[][MAX]){
    if(i == j){
        return 0;
    }
    if(dp[i][j] != -1){
        return dp[i][j];
    }
    dp[i][j] = INF;
    for(int k = i; k < j; k++){
        int cost =
            mcm(p, i, k, dp) +
            mcm(p, k+1, j, dp) +
            p[i-1] * p[k] * p[j];
        dp[i][j] = min(dp[i][j], cost);
    }
    return dp[i][j];
}
```

Fractional Knapsack

```
#include<stdio.h>

struct Item{
    int P;
    int W;
    float R;
};

int partition(struct Item I[], int low, int high){
    struct Item p = I[low];
    int i = low;
    for(int j = low+1; j <= high; j++){
        if(I[j].R > p.R){
            i++;
            struct Item temp = I[j];
            I[j] = I[i];
            I[i] = temp;
        }
    }
    struct Item temp = I[i];
    I[i] = I[low];
    I[low] = temp;
    return i;
}
```

```

}

void quickSort(struct Item I[], int low, int high){
    if(low < high){
        int p = partition(I, low, high);
        quickSort(I, low, p-1);
        quickSort(I, p+1, high);
    }
}

```

Job Sequencing

```

#include<stdio.h>

struct Job{
    int id;
    int deadline;
    int profit;
};

void job(struct Job J[], int d, int n){
    int D[d];
    int Profit = 0;

    for(int i = 0; i < d; i++){
        D[i] = 0;
    }

    for(int i = 0; i < n; i++){
        for(int j = J[i].deadline-1; j >= 0; j--){
            if(D[j] == 0){
                D[j] = J[i].id;
                Profit = Profit + J[i].profit;
                break;
            }
        }
    }
}

```

Activity Selection

```

#include<stdio.h>

struct Act{
    int id;
    int start;
    int finish;
};

void Activity(struct Act A[], int n){
    int Selected[n];

    for(int i = 0; i < n; i++){
        Selected[i] = 0;
    }

    Selected[0] = 1;

    int d = A[0].finish;

    for(int i = 1; i < n; i++){
        if(A[i].start >= d){
            Selected[i] = 1;
        }
    }
}

```

```

        d = A[i].finish;
    }
}

```

Dijkstra

```

#include<stdio.h>

#define MAX 100
#define INF 999999

int minDist(int Dist[], int Vis[], int n){
    int min = INF;
    int index = -1;

    for(int i = 0; i < n; i++){
        if(Vis[i] == 0 && Dist[i] < min){
            min = Dist[i];
            index = i;
        }
    }

    return index;
}

void Dij(int Graph[MAX][MAX], int n, int s){
    int Dist[MAX], Vis[MAX];

    for(int i = 0; i < n; i++){
        Dist[i] = INF;
        Vis[i] = 0;
    }

    Dist[s] = 0;

    for(int i = 0; i < n-1; i++){
        int u = minDist(Dist, Vis, n);

        Vis[u] = 1;

        for(int v = 0; v < n; v++){
            if(
                !Vis[v] &&
                Graph[u][v] &&
                Dist[u] != INF &&
                Dist[u] + Graph[u][v] < Dist[v]
            ){
                Dist[v] = Dist[u] + Graph[u][v];
            }
        }
    }
}

```

Bellman Ford

```

#include<stdio.h>

#define MAX 100
#define INF 999999

struct Edge{
    int src;
    int dest;
    int weight;
};

```

```

void bellmanFord(struct Edge edges[], int V, int E, int s){
    int Dist[MAX];
    for(int i = 0; i < V; i++){
        Dist[i] = INF;
    }
    Dist[s] = 0;
    for(int i = 1; i < V; i++){
        for(int j = 0; j < E; j++){
            int u = edges[j].src;
            int v = edges[j].dest;
            int w = edges[j].weight;

            if(
                Dist[u] != INF &&
                Dist[u] + w < Dist[v]
            ){
                Dist[v] = Dist[u] + w;
            }
        }
    }
}

```